It is important to notice that, because of the complicated decoding procedure illustrated above, the corresponding portfolio trees will not be uniformly distributed over the space of all feasible portfolio trees. However, the evolutionary algorithm is robust enough not to be negatively affected by this initial bias.

Genetic Operators

The same crossover used for one-period portfolio selection is still well suited for the multi-period case. The mutation operator is almost the same as in the one-period problem, except for the fact that there is a difference in how random change is carried out between the first N bytes, encoding for the root portfolio, and the rest of the genotype, reflecting the fact that while the former are a direct encoding of a portfolio, the latter encode for the variation in weight from the parent portfolio.

Therefore, the first N bytes are increased or decreased by one with equal probability, while the remaining bytes are completely overwritten by a new random value from $\{0, \ldots, 255\}$. In other words, the first N bytes mutate very gradually, and the others can mutate more widely.

Fitness and Selection

The fitness of an individual (i.e. of a portfolio tree) is a positive real number f obtained from the objective function z via the transformation

$$f = \begin{cases} \frac{1}{z+1}, & \text{if } z > 0; \\ 1 - z, & \text{if } z \leq 0. \end{cases}$$

Selection is elitist and fitness proportionate, using the roulette-wheel algorithm. Fitness is scaled with the transformation

$$\hat{f} = f - f_{\text{worst}}, \tag{1.37}$$

where f_{worst} is the fitness of the worst individual in the current population.

Overall, the algorithm is elitist, in the sense that the best individual in the population is always passed on unchanged to the next generation, without undergoing crossover or mutation.



Artificial Neural Networks

2.1 Introduction

THE FUNCTIONING of the brain has always fascinated people. The human brain, and also the brain of some animals, is indeed capable of astonishing achievements such as remembering, recognizing patterns, and associating, among many others. The way in which these tasks are performed appears to be quite different in nature from standard computation as we know it, that is, in the Turing sense [146]. Indeed, the brain is a massively parallel, highly connected assemblage of an astronomical number of slow processing units that collectively work on these difficult tasks and allow us to function smoothly and effortlessly. These units or cells are called neurons, they are of several different types, and they work in an analog way by propagating electrical currents of chemical origin along connections. The details of how neurons function are very intricate and need not concern us here but the main points are simple and worth some study. The neuron has three main components: the *soma*, the *dendrites*, and the *axon* (Figure 2.1).

The soma is the cell body containing the cell's nucleus. The axon is a single fiber extending from the cell body and then branching out progressively. The dendrites are shorter and they are arranged in a tree-like fashion around the cell body. The axon branches ter-

Chapter 2

Artificial Neural Networks

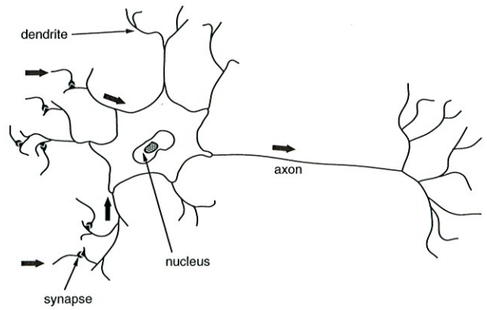


Figure 2.1 A highly stylized form of a biological neuron with its main components highlighted. The arrows show the direction of the electrical signal flow.

minate at the surface of other neurons or on the dendrites. These contact points are called *synapses*. Neurons interact through electrical pulses that propagate along the axons and are transmitted to other neurons via the synaptic connections. If enough incoming pulses arrive at a given neuron in a certain interval of time, the neuron "fires", transmitting a new electrical pulse down its axon. It is this comparatively slow but massively parallel and fault-tolerant signal propagation phenomenon that accounts for the unique properties of the nervous system. The brain is also a low-consumption system, dissipating several orders of magnitude less power than any known digital technology for comparably elementary operations.

The operational properties of the brain arise from several different processes, on different time scales. First of all, the brain has evolved over millions of years and its main structures are coded in the genome. However, given that the human brain has some 10^{11} neurons as well as some 10^{15} connections, the genetic code cannot specify everything from the start. Therefore, the brain shows "plasticity", that is, it continues to modify its connectivity patterns and to adapt during its whole life, as a function of the external stimuli, although plasticity diminishes with age. It is especially the neuron "fanout", that is, the number of direct connections a neuron has with other neurons, that is impressive, this number being typically between 10^3 and 10^5. Just for comparison, this number is of the order of 10 for electronic circuits. No doubt, the highly interconnected nature of the brain plays a major role in the way it functions. The self-organizing and self-adapting capabilities of the brain are also evident in case of impairement of some of the brain area. This also points to another remarkable property of the brain, i.e., its capability to tolerate faults and incomplete information. This fault-tolerance is

difficult to obtain in artificial computational systems and the brain is a precious example to follow in this respect.

Artificial neural networks (ANN) have their origin in the attempt to simulate by mathematical means an idealized form of the elementary processing units in the brain and of their interconnections, signal processing, and self-organization capabilities. The emphasis here is on these models seen as computing systems of a different kind. There also exist computational models of real neurons that are widely used in what is called "computational neuroscience". Computational neuroscience tries to build models that approximate the behavior of actual neurons so as to be able to simulate relatively large pieces of the nervous system. This is useful to complement electrophysiological and other in vivo experiments that are difficult and lengthy to perform. But although these models are very interesting and useful, they fall outside the scope of our subject matter. Instead, we intend to present in this chapter the behavior and properties of highly simplified mathematical abstractions of neurons and of their interactions. We will first outline the simplest formal models of neurons, study their properties and limitations as single units, and then investigate the behavior of networks of such simple processing elements. We will see that neural networks solve problems in very different ways from those we are accustomed to in classical computer science insofar as neural networks approximate answers rather than deterministically compute them. In fact, the usual way to solve a given problem on a digital computer is to provide a series of precise instructions to be followed by the machine, i.e., a "program". On the other hand, a neural network may be viewed as an adaptive system that progressively self-organizes in order to approximate the solution. In other words, neural networks free the problem solver from the need to accurately and unambiguously specify the steps towards the solution. This problem-solving philosophy can be either an advantage or a drawback: it all depends on the application and on the objectives.

Most importantly, we will also see that neural networks have the ability to progressively improve their performance on a given task by somehow "learning" how to do the task better, if given some way of evaluating their current performance, a process in which programming is replaced by learning through examples.

Artificial neural networks are at their best for problems in which there is little or incomplete understanding, so that building a faith-

ful mathematical model is difficult or even impossible, but abundant data is available, since they are data-driven. Problems of this kind are very common in pattern classification, non-linear function approximation and system modeling, control, associative memory, and system prediction among others. ANNs of the kind studied here are related to traditional mathematical and statistical models, as we will see in this chapter.

Although this chapter provides a reasonably detailed account of the most common ANN types and gives an adequate background for the rest of the book, it is far from exhaustive. We have concentrated on ANN models that can be used in conjunction with other soft computing techniques, and these combinations will be the subject of subsequent chapters. The subject of ANNs has grown into quite a large field of study and more complete descriptions can be found in a number of books. Among those we can recommend Gurney's book [82] at an elementary level and Hassoun's [86] for a deeper and more complete presentation.

2.2 Artificial Neurons

BEFORE studying assemblages of artificial neurons it is sensible to describe the properties of isolated units. The first model of a formal neuron was proposed by McCulloch and Pitts in their landmark 1943 paper [139] and is called a *Threshold Logic Unit* (TLU) or a *Linear Threshold Gate*. Figure 2.2 gives a graphical representation of such a unit with n real-valued inputs x_i each input being associated with a parameter w_i. Parameter w_i is also known as a "synaptic weight", or simply "weight", in analogy with biological synapses, the functional contacts between two nerve cells. A TLU performs a weighted sum operation followed by a non-linear thresholding operation, or step function, such that if the value of the sum is greater or equal than the *threshold* θ then the output y of the unit is 1, otherwise it is 0:

$$y(x) = \begin{cases} 1 & if \ \sum_{i=1}^{n} w_i x_i \geq \theta, \\ 0 & otherwise \end{cases} \tag{2.1}$$

In other words, the neuron will "fire" that is, it will emit an instantaneous "1" signal if the thereshold is exceeded; otherwise, it will do nothing. The weighted sum of equation 2.1, also called the neuron

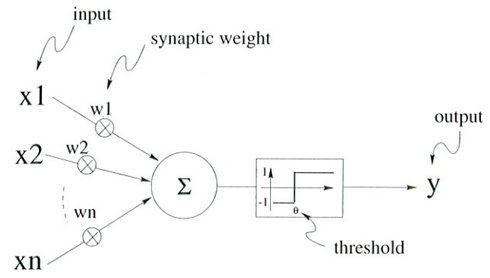


Figure 2.2 The thresholding logic unit of McCulloch and Pitts.

activation, can be expressed more concisely as the scalar product $\mathbf{w} \cdot \mathbf{x}$ of the weights vector $\mathbf{w}$ and the input vector $\mathbf{x}$.

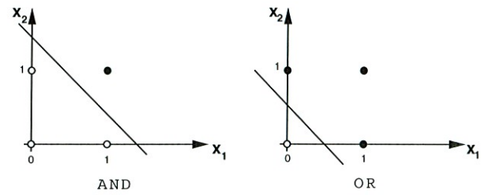


Figure 2.3 Linear separations of input space corresponding to the AND and OR logic functions.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR truth table

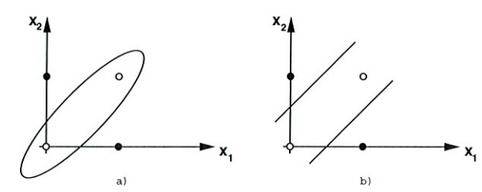


Figure 2.4 XOR implementation with (a) a nonlinear separation of the input space, and (b) two linear separations.

Thus, a TLU performs a mapping $\mathbb{R}^n \rightarrow \{0, 1\}$ from the reals to Boolean values. If the inputs are binary, that is if the n-component vector $\mathbf{x} \in \{0, 1\}^n$ then a TLU becomes a boolean function. It would then be interesting to know the computational power of TLUs, i.e. can a TLU realize all the 2^{2^n} possible boolean functions of n inputs? The global answer is that a single TLU can only realize a subset of all those functions (for more details see [86]). The functions

that can be attained by a TLU are called *linearly separable* which means that they are able to categorize the inputs in two classes which are separated by a hyperplane in an n-dimensional boolean space. For instance, among the familiar boolean functions with two inputs, AND and OR are linearly separable, whereas the boolean equality and inequality (also called XOR or exclusive OR) are not. This can be seen pictorially in Figure 2.3 and Figure 2.4. In two dimensions a hyperplane becomes a straight line and there is no way of drawing the line in order to separate the two different classes in the XOR case. However, it can also be seen in this last case that a non-linear separator or a couple of straight lines can perform the classification task.

2.3 Networks of Artificial Neurons

AS WE HAVE SEEN , artificial neurons in isolation are not very impressive. However, the living example of the nervous system shows that large assemblages of simple cells give raise to astonishingly complex emergent behaviour. Likewise, it is only when many artificial simple units are brought together to form a network that useful new computational capabilities appear. In this section we will study such artificial neural networks and some of their properties.

Although we have just seen that a single TLU is unable to represent all the Boolean functions, it can nevertheless realize the NAND and NOR gates. Since NAND (and NOR) are universal logic gates, it follows that a TLU is also universal and any Boolean function can be realized by a suitable network of TLUs. This result had already been recognized in the classic paper by McCulloch and Pitts [139]. There exist many kinds of neural network architectures, some of which are depicted in Figure 2.7. Among these the *feedforward multilayer networks* have received much attention due to their relative simplicity and computational capabilities. In feedforward networks an input pattern is transformed to an output pattern through a finite series of layers of nodes, some of which may have no connections to the input or output, and there are no feedback signals i.e., signals only travel forward (see Figure 2.7(b)).

For instance, it can be shown that the feedforward two-layer network of Figure 2.5 solves the XOR classification problem by implementing two linear decision boundaries, as depicted in Figure 2.4.

The intermediate node layers are called *hidden* nodes because they are not directly connected to the external world through the inputs and the outputs.

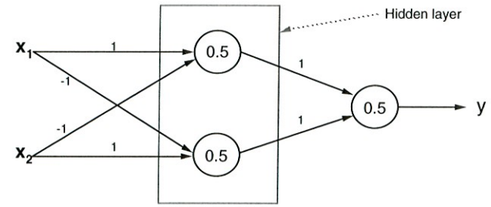


Figure 2.5 Multilayer feedforward network.

Until now we have been mainly talking about Boolean functions as the use of a threshold function limits the output to be 0 or 1. However, it is possible to make the artificial neuron emit a continuous signal. This can be obtained by using a continuous transfer function following the weighted sum instead of the discontinuous step function, as represented in the following equation:

$$y(x) = g\,(\sum_{i=0}^{n} w_i x_i), \tag{2.2}$$

where $g(w_i x_i)$ is called the *activation function*. A convenient function form for g is the so-called *sigmoid* which is graphically represented in Figure 2.6 and whose analytic form is:

$$y(x) = \frac{1}{1 + e^{-(bx-c)}} \tag{2.3}$$

Other commonly used continuous transfer functions are linear, hyperbolic tangent, and gaussian. The sigmoid is suitable because it is differentiable and it saturates i.e. it tends asymptotically to 0 and 1 at the extremes. By varying the parameter of the exponential one can control the steepness of the curve and in the limit it reduces to a TLU. Note that still other functions can be used provided that they are continuous, monotonically increasing, and take values between 0 and 1.

Networks of neurons with real-valued inputs and a sigmoid transfer function can be used to approximate mathematical functions. In fact, it is possible to think of a network of such units as implementing a mathematical function of its inputs. This allows the parameterization of an unknown real-valued function which is very useful in situations where the exact form of the functional relation (if any)

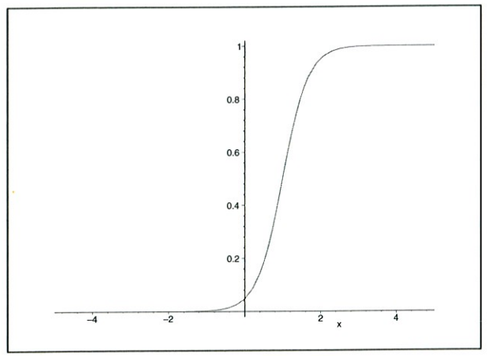


Figure 2.6 A typical sigmoid activation function.

is unknown, such as time-series prediction or system identification. More generally, let us suppose that we want to approximate any continuous real-valued function $F(x_1, x_2, \ldots, x_p)$. A general result (see for instance [86] and references therein) says that such a family of functions can be approximated to any desired accuracy by a feed-forward network with at least one single hidden layer. That is, there exist real constants α_i, w_{ij} and a monotone-increasing continuous function g (e.g. the sigmoid function) such that:

$$f(x_1, x_2, \ldots, x_p) = \sum_{k=0}^{p} \alpha_k \, g(\sum_{j=0}^{m} w_{ij} v_j) \tag{2.4}$$

and

$$\mid f(x_1, x_2, \ldots, x_p) - F(x_1, x_2, \ldots, x_p) \mid < \epsilon \tag{2.5}$$

for any $\epsilon > 0$, where m and p are the number of units in the input and hidden layers respectively. Being only an existence result, the theorem leaves unspecified the optimum number of hidden layers and the number of units in the hidden layers in any given case. A related result is that single-hidden-layer nets with sigmoidal activation units and a single linear output node are universal classifiers, i.e. they can correctly assign any given input pattern to one of a finite number of distinct classes. In practice, networks used for classification usually

employ several output units, each of which represents a unique class. This setting is more natural, as it avoids the use of a large number of hidden units.

Thus, ANNs possess interesting computational capabilities. But the remaining problem is that of designing the networks in such a way that a given computational task can be realized as efficiently and economically as possible. How should we go about designing a neural network? How many units and layers should we use? How to choose the connections and their weights? There are no easy answers to these questions and no standard design recipes exist for designing neural networks for a given problem. In general, neural network researchers prefer to let the network design itself, so to speak, rather than imposing a pre-existing architecture. This is indeed a very peculiar way of tackling a computational problem and it will be dealt with in the following section.

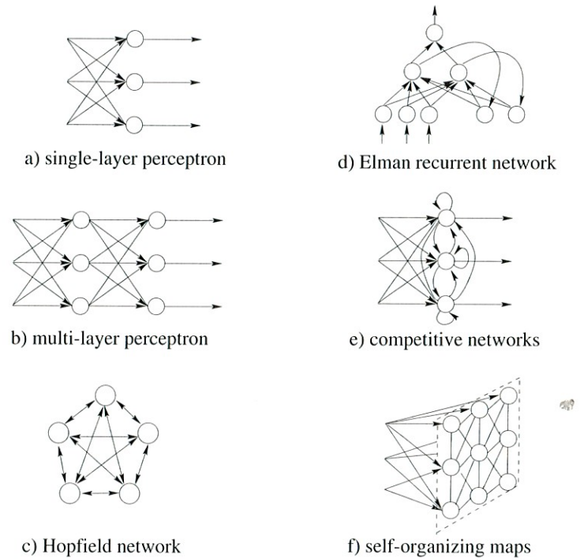


Figure 2.7 Typical neural network topologies.

2.4 Neural Learning

WE SAW that one of the distictive features of artificial neural networks is the absence of a pre-defined set of instructions to follow. Instead, neural networks represent a different conceptual approach to computation that makes use of statistical concepts such as progressively improving input-output relationships by a process in many ways analogous to function fitting and extrapolation. In this way, the network adapts itself to perform a given task. In the neural network literature such a process goes under the name of "learning" or "training". Perhaps, terms such as "learning" are misleading since they may suggest that a phenomenon similar in complexity to human or animal learning is taking place. Indeed, nothing of the sort is actually occurring when a network is trained to perform a given task. The process is entirely of a formal nature and can be described by well-defined mathematical algorithms. Nevertheless, network and machine learning is such a widespread concept that the term will be used here in this restricted sense.

In its most common meaning, learning by a neural network implies an adaptive procedure in which the weights of the network are incrementally modified so as to improve a prespecified performance criterion i.e., an objective function over time. Such a procedure is called a *learnig rule* or *learning algorithm* and the adaptation may take place in a *supervised* or an *unsupervised* way. In supervised learning the net is presented with a set of known input/ouput pattern pairs: this set of values is called the *training set*. The learning process consists in updating the weights at each training step so that, for a given input, an error measure between the network's output and the known target value is reduced. This is also known as learning with a teacher or associative learning.

In unsupervised learning there is still an input/output relationship that the network must learn to reproduce but no feedback is provided indicating whether a given association is correct or not. In other words, the similarities among patterns and features of a training set must be discovered by the network itself by *clustering* similar cases in a *self-organizing* manner.

A third commonly used form of training a network makes use of the concept of *reinforcement learning*. In this case the "teacher" signal for some training input/output pair is not some measure of the difference between the given output and the expected value, as

in supervised learning, but rather an evaluation of the result as a "wrong" or "right" direction.

In the following sections some of the most important supervised and unsupervised learning algorithms will be briefly described. Since the subject is a large one, we cannot hope for complete coverage: for an in-depth treatment of the subject, the reader can consult for instance reference [86].

2.5 Supervised Learning

SUPERVISED learning algorithms are based on error correction rules, that is, an error value is generated from the actual response of the network and the desired response. Following that, the weights are modified such that the error is gradually reduced.

We start by describing some simple single-neuron training rules and then go on to training algorithms for networks of units.

2.5.1 Perceptron Learning Algorithm

A classical and very simple example of supervised learning is Rosenblatt's *perceptron learning algorithm* [190]. For our purpose here, a perceptron is a binary unit similar to the linear threshold gate of Figure 2.2. The outline of the learning algorithm is given in Figure 2.8.

1. Initialize weights and threshold randomly,
2. Present an input vector to the neuron,
3. Evaluate the output of the neuron,
4. Evaluate the error of the neuron and update the weights according to:
   $w_i^{t+1} = w_i^t + \eta(d - y)x_i$,
   where, d is the desired output, y is the actual output of the neuron, and η $(0 < \eta < 1)$ is a parameter called the *step size*.
5. Go to step 2 for a certain number of iterations, or until the error is less than a prespecified value.

Figure 2.8 Perceptron learning algorithm.

Historically, the algorithm is called the perceptron learning rule because it was first used by Rosenblatt with a variant of the TLU called a perceptron. In this algorithm, the positive parameter η is called the *learning rate* or step size: it dictates the size of the weight change and thus it controls how fast the learning takes place.

The rule has a straightforward geometric interpretation in terms of vector algebra in $\mathbb{R}^n$ but we do not have space here to go into the details. To give a feeling for how the algorithm works, let us consider a decision problem in which the inputs must be partitioned into two classes. In the perceptron context, learning is equivalent to changing the vector of weights in such a way that a decision surface moves until a hyperplane is found that correctly separates all input/output pairs into two distinct classes. The interested reader can consult [82] for a more formal explanation.

Rosenblatt proved that if the two classes of input patterns are linearly separable, then the perceptron algorithm will eventually converge after a finite number of iterations. This is known as the *perceptron convergence theorem*. An important property of the perceptron is that whatever it can compute, it can learn with the perceptron learning algorithm. However, if a given problem is not linearly separable, a perceptron does not find a low-error solution, even if it exists.

The perceptron learning rule is of historical interest but it fails on nonlineraly separable problems. We have seen in Section 2.3 that such nonlinearly separable problems can be solved in principle by multi-layer nets of LTUs. However, it turns out that multi-layer nets cannot be trained effectively by the perceptron learning algorithm. For training purposes, it has been found that instead of using straight weight adjustment, it is more practical to use gradient-based descent on an appropriate differentiable criterion function. For instance, we could use as an objective function to be minimized the sum of the differences between the actual neuron output and the target training output values. This amounts to finding the minimum of the sum of errors over the training set as a function of the free weight parameters. Normally, the square of the sum of the errors is used in order to work with positive quantities. Because of the discontinuous nature of the output step function, the gradient, and thus the derivatives of the criterion functions are also discontinuous. This is easy to fix by just considering the neuron activation i.e., the weighted sum of inputs instead of the output value. As we will see, this form of

training is suitable for multi-neuron, multi-layer networks, allowing those networks to solve arbitrary classification problems (see also Section 2.3).

2.5.2 LMS Rule and Delta Rule

The LMS (*Least Mean Square*) algorithm by B. Widrow and M. Hoff [245] was developed to provide "learning" to a McCulloch-Pitts-like element called ADALINE for ADAptive LINear Element. Such nodes are identical to TLUs except that the output signals are $\{-1, +1\}$ instead of $\{0, 1\}$. It uses a hard-limiting function with a bias weight w_0 controlling the threshold level of such a function. The learning rule tries to reduce the mean square error (MSE), averaged over the training set, using the *gradient descent* method. Here the criterion function to be minimized is:

$$E(\mathbf{w}) = 1/2 \sum_{i=1}^{m} (d_i - y_i)^2 \tag{2.6}$$

That is, the sum of squared errors over m training pairs, where $\mathbf{w}$ is the vector of weights. Taking the derivatives gives us the iterative form of the rule:

$$\mathbf{w}^{t+1} = \mathbf{w}^t + \mu \, (d_k^t - y_k^t) \, \mathbf{x}_k^t, \tag{2.7}$$

where μ is a parameter that controls stability and rate of convergence, $\mathbf{x}$ and $\mathbf{w}$ are the input and weight vectors respectively, y is the output value, d is the expected value and t and $t+1$ are the current and next step in the iteration respectively.

In the LMS algorithm the weights are updated using an estimate of the steepest descent of the mean square error function $E(\mathbf{x})$ in weight space. E is a quadratic function of the weights and is therefore convex and has a unique (global) minimum. If the chosen positive constant μ is sufficientely small, the gradient descent search will asymptotically converge toward the solution regardless of the initial weight values. Widrow extended his ADALINE units to multiple Adaline networks and provided one of the first trainable layered networks.

The so-called *Delta* rule is similar to the LMS rule: it also works by minimizing an error residual but now the activation function of the neuron unit is a continuous, differentiable, non-linear curve such

as a sigmoid. Gradient methods for minimization can be used with the Delta rule and the output value can appear in the quadratic criterion function since the transfer function is differentiable. The convergence speed of the Delta algorithm depends on the slope of the first derivative of the transfer function, which can be rather flat at the extremes. In these regions progress is very slow but convergence can be improved in various ways (see for instance [86]).

The importance of the Delta rule is due to the fact that it extends naturally to the training of multilayer networks by a method called *error backpropagation*. Backpropagation was worked out early in the 1970s by Paul J. Werbos. The next section gives an account of this important technique.

2.5.3 Backpropagation Algorithm

The absence of a practical method for training multilayer networks nearly stopped work on ANNs for several years. The invention of the backpropagation method, attributed to Paul J. Werbos [239], is one of the main reasons for the renewed interest in artificial neural networks toward the end of the 1970s.

The algorithm is by far the most popular method for performing supervised learning of feedforward networks composed of continuous activation function units such as those described in the previous paragraph on the Delta rule in order to be able to use derivatives for calculating the gradient. It has been used on countless applications of ANNs to many different kinds of problems. Error backpropagation is essentially a search procedure that attempts to minimize a whole network error function such as the sum E of the squared error of the network output over an ensemble of training input/output pairs:

$$E = 1/2 \sum_{j=1}^{m} (d_j - y_j)^2, \tag{2.8}$$

where d_j is the desired jth output and y_j is the actual jth output of the network

The name of the algorithm is due to the fact that the weight modifications dictated by the learning rule propagates "backwards" from the output layer to the input layer. In fact, the algorithm can be intuitively visualized as a series of forward and backward waves of activity. In the forward phase the network produces its output for

given input/output pairs: this leads to the calculation of the global error. In the backwards phase, the weights of the output nodes and then those of the hidden nodes back to the input, are modified layer by layer according to the learning rule in order to reduce the error. The details of the algorithm are mathematically simple but require some space to be presented in an orderly way. We give here only an outline of the basic procedure (Figure 2.9). An exhaustive description can be found in [86]. An excellent pictorial explanation is contained in [188].

1. Initialize weights randomly,
2. Present an input vector pattern to the network,
3. Evaluate the outputs of the network by propagating signals forwards,
4. For all output neurons calculate $\delta_j = (y_j - d_j)$,
   where d_j is the desired output of neuron j and
   y_j is its current output:
   $y_j = g(\sum_i w_{ij} x_i) = (1 + e^{-\sum_i w_{ij} x_i})^{-1}$,
   assuming a sigmoid activation function,
5. For all other neurons (from last hidden layer to first),
   compute $\delta_j = \sum_k w_{jk} g'(x) \delta_k$,
   where δ_k is the δ_j of the succeeding layer,
   and $g'(x) = y_k(1 - y_k)$,
6. Update the weights according to:
   $w_{ij}(t+1) = w_{ij}(t) - \eta y_i y_j (1 - y_j) \delta_j$,
   where, η is a parameter called the *learning rate*.
7. Go to step 2 for a certain number of iterations, or until the error is less than a prespecified value.

Figure 2.9
Backpropagation learning algorithm.

Remarks on the Backpropagation Algorithm

Today there exist many public domain or commercial packages implementing some form of feedforward ANNs with backpropagation as the learning algorithm. There is nothing wrong with this "canned" software as many of them are of high quality and the method is quite standard and well-known. Indeed, in view of the number of applications that use supervised neural learning, it would not make much sense to reinvent the algorithm each time. Nevertheless, there are a number of subtle points in training ANNs

that make life less pleasant and that should at least be briefly mentioned.

Local and Global Minima

First of all, consider that backpropagation essentially does a gradient descent search through the multidimensional space of possible weights trying to reduce the error E between the training output values and the network outputs. Till now, we have not explicitly mentioned the fact that the search hypersurface will almost surely be multi-modal i.e., it may present a number of minima. In minimizing $E(\mathbf{w})$ for such a function we would like to strive for the *global* minimum, that is the value $\mathbf{w}^*$ of the weight vector for which $E(\mathbf{w}^*) < E(\mathbf{w})$ for any $\mathbf{w}$ in the search domain. Gradient descent is a minimization algorithm that will find the closest local minimum with respect to the starting point of the search, and if the search landscape is complex enough, this minimum is very unlikely to be the global one. What does this mean in terms of the learning process? Two things can be said in general about this state of affairs: first, the problem of the search getting stuck in a local minimum is not as important in practice as one might think. Network weights determined in this way have proved to be good enough in many applications. Secondly, there exist other optimization methods that can help the search to escape local minima. For example, evolutionary algorithms (see Chapter 1), which can be seen as biased stochastic optimizers, can be used to train neural networks of various kinds. This is a first example of the synergy between different soft computing methodologies which is one of the leading themes of our book. Other global optimization methods, such as stochastic gradient search or simulated annealing can also be used. Another problem with the standard backpropagation algorithm is that it can be very slow. But researchers have found a number of ways to speed-up and enhance backpropagation learning (see for instance [86]).

Learning Topologies

Another issue which we have largely ignored until now is the question of network topology. That is, backpropagation or similar learning algorithms are customarily used simply for adjusting the weights in an otherwise fixed feedforward network structure. But it is clear that the interconnection of the units and their number plays an important role. In fact, lack of knowledge in determining the appropriate topology for a problem, including the number of layers, the number of neurons per layer, and the interconnection scheme, often results in slow learning speed and poor performance. For example, if a network is too small, it does not accurately approximate the desired input to output mapping. If a network is too large it requires longer training times and may not perform well on unseen data. A variety of methods have been proposed to dynamically grow or prune the number of neurons and interconnections in order to improve the performance of the network. For example, the *cascade-correlation* algorithm starts with a network with no hidden units and keeps adding units one at a time, choosing the weight such that the residual error is minimized [60]. One can also go the other way around, starting with a complex network and shrinking it by pruning units away when it is found that certain connections have a small influence on the network error. Some of the methods in this class alter the topology while at the same time using backpropagation of the output errors, some do not; but we do not have space here to go into the details, for which the reader can consult [10] for example.

We note here that one major application of evolutionary algorithms to ANNs has been evolving neural network topologies. This topic, together with other interactions between ANNs and EAs will be the theme of Chapter 4.

Generalization and Overfitting

Generalization means the ability of a learning system to correctly map new inputs that were not previously used in the training phase. If we let the network adapt too well to the training data by giving it too many degrees of freedom, then the residual error will be very small but the network is likely to fail to correctly map new, previously unseen input data of the same class i.e., it will have poor generalization capabilities (see Figure 2.10). This phenomenon is known in the statistical and ANN literature as *overfitting* the data and should obviously be avoided as far as possible. The problem is that the net adapts too much to the noise and fine behaviour present in the data while ignoring the underlying general trends.

What can be done to improve the generalization ability of a neural network? One well-known technique comes from regression analysis in statistics and consists in dividing the training data into two sets:

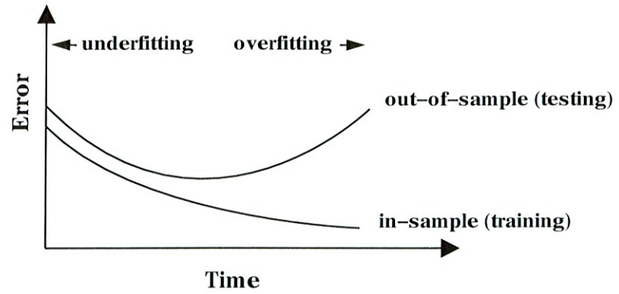


Figure 2.10 Illustration of overfitting. Too much training may lead to small residual error but with bad generalization performance.

a training set proper and a *validation* set. The training set is used for learning in the usual way but from time to time the validation set is used to test out-of-sample performance. This cross-validation procedure continues until the error on the validation set starts to increase, for this means that the generalization performance attained is good. Ideally, the best model is the one at the minimum of the validation set error curve. Cross validation is a useful empirical technique that allows us to distinguish between signal and noise in the absence of further information, and its proper use is the subject of much discussion.

The appropriate number of hidden units and the number of training data are also related and play a role in the quality of the resulting network. There are no rigorous results here but a rule of thumb says that the number of hidden units needed will increase as the number of training data increases for a given level of performance. The techniques outlined in the previous section may help in finding a nearly optimal or satisfactory net topology for a given task.

2.5.4 Applications of Supervised Learning Networks

Supervised neural networks, especially feedforward nets trained by backpropagation, have been used in a wide range of applications including signal and image processing, speech recognition, system identification, medical diagnosis, financial prediction, detection of events in high-energy physics, etc.

It would be inappropriate to treat representative applications in detail in this introductory chapter. However, a brief description of a

couple of typical areas in which ANNs have been successful should help the reader by giving a flavor of the work that has been achieved.

An Illustrative Example: Character Recognition

Fast and reliable character recognition, especially of hand-written characters is extremely important in practice. Think for instance of pen-based palm organizers, of automatic address reading and routing of postal mail and of check signatures, just to mention a few. There have been several attempts to solve this problem by using feedforward ANNs. It is very easy to find plenty of training data in this field but the task is a very difficult one even for human recognizers, not to speak of computer programs. Given the difficulties of the problem, no classification system is expected to achieve a success rate of 100 percent. Hundred percent success is not really needed for the system to be useful, but too low a recognition rate, say under 95%, will result in a system that is not reliable enough to be usable in practical applications. An example of this was the Apple Newton, a palmtop computer that employed handwriting input and recognition, but in which error rates were sufficiently high as to seriously limit the user acceptance of the device.

The main ideas in using feedforward nets for character recognition are simple enough: once one has obtained the hand-written data, the characters are scanned and digitized as a gray-scale pattern represented as a grid of pixels of a certain size. Two examples of such a grid are schematically depicted in Figure 2.11. Note, though, that actual grids are likely to have many more pixels in them.

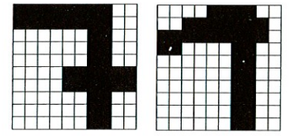


Figure 2.11 An array of pixels representation of two digits. Left hand side: the idealized digit 7. Right hand side: a more likely example of a handwritten or noisy 7.

For characters to be interpreted on personal digital assistants, the recording is made online using a special pad which is able to sense the pen's pressure and velocity. The data are then divided into a learning and a test set. The learning set is presented to the network one image at a time and the error in classification is progressively reduced by using backpropagation. When an acceptable error level has been attained and misclassifications are rare, the net is used to classify the yet unseen examples. The structure of an idealized ANN

architecture for decimal digit recognition is shown in Figure 2.12. The inputs are the pixels representing a single character and the outputs are the classes $0-9$ into which the given character is to be classified.

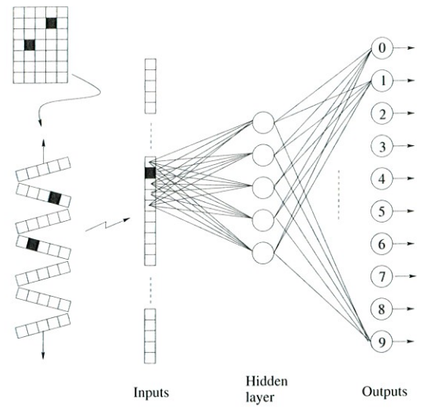


Figure 2.12 A schematic network architecture for character classification.

For example, in a well-known application to handwritten numerical ZIP codes appearing on U.S. mail, a feedforward net was trained to learn to recognize handwritten digits [49]. The ANN learned on several thousand samples and was tested on about two thousand unseen cases with good results. The original network architecture is actually more complex than what one could imagine from our short description. There were a total of 1000 units of which ten were the output units corresponding to the classification of the digits into one of ten groups $(0-9)$. The network input was a 16×16 array containing the gray-level pixel image of a particular handwritten digit. There were three hidden layers of which the first two were able to extract smaller image features, while the third was a standard fully connected layer. The net was trained by an accelerated version of backpropagation. As a refinement of the above work, LeCun *et al.* [49] did weight pruning using statistical methods in order to make a nearly optimal selection of weights with the goal of increasing the generalization capability of the network. This resulted in better overall performance of the system. For more recent discussions of neural and fuzzy approaches to handwriting recognition see [73, 45].

A Second Example: Financial Markets Forecasting

The dynamics of financial markets is poorly understood. Although market data are generally and publicly available, nobody really knows how market prices will evolve in spite of claims to the contrary. The fact is that financial markets are very complex evolving systems of many interacting agents that have their own beliefs, expectations, and trading strategies. Apart from textbooks describing extremely simple and idealized cases, there is absolutely no market model available. Again, when there are no models but empirical data abound, ANNs, among other techniques, could be employed to approximate the unknown relationships underlying the observed values.

Being able to predict market evolution, at least the trend during a specific time window, is obviously a definite advantage for traders. Some people believe that the future behaviour of a given market is somehow "implied" by the current and past market data and that the careful study of these data patterns will reveal future trends to some extent. Others think that only the fundamental economic indicators such as a nation's GNP (Gross National Product), unemployment rate, and the like have an influence on markets. Yet others think that market indicators such as the yen/dollar ratio are the really important data to take into account. Finally, there are those for whom the market is not predictable at all, not even in principle. For them, prices essentially follow a random walk and any information is immediately absorbed into the current prices and cannot be made use of.

Today, there are technical reasons to believe that markets are not entirely random, at least on some time scales, although the question is far from settled [58]. If one accepts this view that there are opportunities for predicting future prices, what are the possible approaches? One could use econometric models such auto-regressive moving averages of prices plus random shocks to try to forecast future behaviour. But these models are normally linear ones, while non-linear and even chaotic behaviour has been reported for financial markets [58]. Thus, it would seem that feedforward multilayer ANNs might be useful, since they are able to map any well-behaved function of the inputs thanks to their universal approximation capabilities (see Section 2.3).

Feedforward networks trained by backpropagation can be employed to achieve non-linear forecasting of time series. The general

idea exploits a result of Takens [86] and it goes as follows: let

$$p(t), p(t-1), p(t-2) \ldots p(t-m) \tag{2.9}$$

be the current and past $m+1$ values of an observation such as a price or the sampled value of a signal. Using these values as an input pattern to a neural network and the observed value $p(t+1)$ at time step $t+1$ as a target for many values of t, we can train a feedforward network to make one-step ahead prediction of the unknown future value of the time series. Prediction n steps ahead can also be done but the quality of the prediction will degrade as n increases. Once the net has learned to make the prediction on the training values, it can be used out-of-sample for predicting future values of the series. What the network learns implicitly is of course an approximation of the unknown functional relationship between the next value and the m past samples of the variable. Reported results using this prediction method are at least as good as several other methods for non-linear forecasting [238]. This univariate, pure time-lagged model works well for physical systems showing well-defined deterministic chaos. In the less precise world of economical prediction one could also exploit the multivariate approach that consists in taking into account related time series to the one for which a predictor is sought and also some important market indicators as input values. Such an approach has been implemented with good effect in [6, 7], where a feedforward neural network is used to find a mapping f of the form:

$$x(t) = f(I_1, I_2, \ldots, I_n, x(t-\Delta_i)) \tag{2.10}$$

where $x(t-\Delta_i)$ are lagged values of the variable to be predicted and the I_j are financial indicators, possibly lagged, that might have an influence on x according to economic theory. Among the many possible indicators, the most influential ones are chosen by using statistical sensitivity analysis methods. For instance, in a prediction study of the Swiss stock market index (SPI) [6], it was found that the most relevant indicators are the monthly differences of long interest rates in Germany and Japan and the money exchange rate between USA and Japan. These indicators, together with the past samples of the SPI time series are fed as inputs to a two-layer network. The output is the one-step ahead prediction of the SPI value.

Fairly good results have been obtained with ANNs in the financial markets, in general better than those obtained with most other

methodologies (see for instance [58] and references therein). However, one should be aware that there are no "magic" solutions in this difficult field, only promising techniques. The phenomena are not well understood and the ANN approach, even when it works, has a limited temporal validity due to the evolutionary character of the markets. Besides, the well-known black-box character of ANNs calls for using them together with other methodologies having more explanatory power.

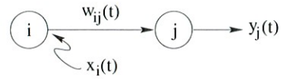
2.6 Unsupervised Learning

IN PREVIOUS sections we discussed networks of units in which the nets were trained to perform an unknown input/ouput mapping by presenting the network examples of input/output pairs. In unsupervised learning there is no feedback from the environment as to the correcteness of the mapping; in other words, there is no "teacher". Instead, the network must be able to discover by itself any categories, patterns, or features possibly present in the data. Networks that are able to infer pattern relationships without being supervised are also called *self-organizing*. Among the unsupervised learning rules, we will briefly discuss here Hebbian learning for single units and networks, competitive learning, and Kohonen's self-organizing feature maps.

2.6.1 Hebbian Learning

The rule that goes under his name, was proposed by Donald O. Hebb in his seminal work *"The Organization of Behavior"* [87]. Hebb's rule makes the weight strength proportional to the product of the firing rates of the two interconnected neurons. That is, when two connected neurons fire at the same time and repeatedly, the synapse's strength is increased. This biologically-motivated rule can be expressed for a single unit as :

where x_i^t and y_j^t are the output values of neurons i and j at time t, w_{ij}^t is the current interconnection weight between neuron i and j, and ρ is a parameter called the *learning rate*. w_{ij}^{t+1} is the future value of the synaptic weight being updated during learning. Hebb's rule, which has inspired a large number of learning algorithms has the



$$w_{ij}^{t+1} = w_{ij}^t + \rho\, y_j^t\, x_i^t, \tag{2.11}$$

important characteristic that it is local. The original Hebb rule is divergent, progressively driving the weight to an infinite magnitude. Several modified Hebbian rules that do not have this undesirable property have been devised [86].

Hebbian learning can be applied to networks of interacting units. One of the most studied approaches is called Principal Component Analysis (PCA). PCA is a standard statistical technique whose goal it is to extract m normalized orthogonal vectors in the input space that account for as much of the data's variance as possible. By projecting the n-dimensional input vectors onto the m ($m < n$) orthogonal directions, we can achieve dimensionality reduction with minimum loss in information content. Such a transformation is achieved by rotating the coordinate system with standard linear algebra operations. E. Oja [160] demonstrated that an artificial neural network is able to compute in parallel, and on-line, the PCA transform. The PCA network is a layer of parallel linear artificial neurons. Oja's algorithm is a modification of Hebb's rule in which a weight-decay term proportional to y^2 has been added. In Oja's network PCA takes place as a consequence of unsupervised neural learning with a Hebb-like learning rule in which weight updating for a network is done as follows:

$$w_{ij}^{t+1} = w_{ij}^t + \rho\,(x_j^t - \sum_{k=1}^{m} w_{kj}^t y_k^t)\, y_i^t, \tag{2.12}$$

where ρ is a learning constant and the other symbols have the usual meaning. There is not room to further develop these ideas here but the interested reader can consult [86] for instance.

2.6.2 Competitive Learning

Competitive learning is an unsupervised learning procedure in which the neurons of a network learn to recognize clusters of similar input vectors. The network detects regularities and correlations among the input vectors and adapts the future response of the units to similar inputs. In competitive networks output units compete among themselves for activation. The simplest competitive learning network consists of a single layer of output neurons to which all inputs are connected. All the units are presented with a given input vector $\mathbf{x}$ but only one output neuron is activated at any given time: the so-called *winner* neuron. The winner unit i is the one with the largest activation value (see Section 2.2):

$$\mathbf{w}_i \cdot \mathbf{x} \geq \mathbf{w}_k \cdot \mathbf{x}, \quad \forall k \neq i \tag{2.13}$$

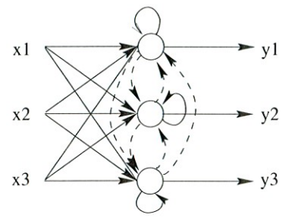


Figure 2.13 A single layer competitive learning network. The solid lines indicate excitatory connections whereas the dashed lines indicate inhibitory connections.

If the input vectors are normalized ($\| \mathbf{x}_k \| = 1$, for all $k = 1 \ldots n$) then the unit i with the smallest activation:

$$\| \mathbf{w}_i - \mathbf{x} \| \leq \| \mathbf{w}_k - \mathbf{x} \|, \quad \forall k \neq i \tag{2.14}$$

that is, the unit with normalized weight closest to the input vector, becomes the winner.

The winner-take-all operation is implemented by connecting the outputs to the other neurons by means of so-called *lateral inhibitions*, and also by means of *self-excitatory* connections as depicted in Figure 2.13. Reference [86] and the references cited therein give details of how inhibition and self-excitation is effected.

In practice, there is no need for the above net to be implemented explicitly: the winning neuron can be found by simple search of the maximum activation. The neuron having the maximum activation updates its weight while the weights of the other neurons remain

unchanged according to the iterative application of the following rule:

$$\Delta w_{ij} = \begin{cases} \rho(x_j - w_{ij}) & \text{if } i \text{ is the winning unit} \\ 0 & \text{otherwise} \end{cases} \tag{2.15}$$

The training process can be seen as a progressive tilt of the weight vector of the winning unit towards the direction of the current input vector. Normalized input vectors (see above) can be depicted as points on the surface of a unit sphere (or circle). At the beginning the unit's weight vectors and the input data vectors are not aligned, if one assumes that both the initial weights as well as the input values are drawn from a random distribution. With time, the competitive learning rule tends to associate certain units with neighboring input vectors, also called data *clusters*. This phenomenon is geometrically represented in Figure 2.14. When a stable solution is found then the weight vector for each cluster represents in some sense the "center of gravity" of that cluster, a kind of typical vector for this class of data. Since the number of data clusters is not known in advance, some units in the network, that happen to be far from any input vector, will turn out to be useless, in the sense that their activation is small or zero for the data of the training set. This is what happens to vector $\mathbf{w}_4$ in Figure 2.14. These "spare" units could still be useful if the distribution of input vectors changes in time, making the system more robust. Otherwise, there are several ways to turn these units into "useful" units during the learning process (see [86]).

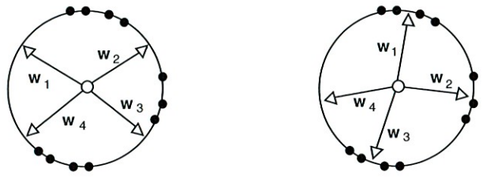


Figure 2.14 A schematic representation of unsupervised competitive learning. The points on the circle surface represent training data. On the left hand side of the figure initial unit weights are represented as random vectors. After training, the weight vectors orient themselves toward the data clusters (right hand side figure).

Vector Quantization

An important use of competitive learning networks is *vector quantization* which has applications in data compression. In this scheme the input space is divided into disjoints regions in such a way that any input vector $\mathbf{x}$ falling into one of the regions is represented by a single label characterizing that class. The class label encoding can then be used later instead of the vector itself, leading to efficient compression of data for storage and transmission purposes. The classes are defined by a number of prototype vectors and the class to which a given input belongs is calculated by taking the nearest prototype vector using a Euclidean distance metric.

There also exist supervised versions of vector quantization, such as Kohonen's *learning vector quantization* [86].

2.6.3 Self-Organizing Features Maps

Biological neural networks often show an architecture that depends on their function. For example, in the visual region of the mammalian cortex the receptive zones are arranged in such a way that neighboring light photons will stimulate areas of the cortex that are also physically close to each other. This is called a *topographic map* and is quite a common arrangement of neurons in regions of the cortex that are responsible for processing sensory information such as visual and audio stimuli. Several observations point to the fact that such topographic maps are not entirely genetically determined. Rather, they seem to be created during the individual development by a sort of unsupervised learning process.

Teuvo Kohonen invented an ANN model directly inspired by the existence of such topographic maps in the brain [114]. Kohonen's self-organizing map (SOM) is an unsupervised learning network in which the neurons have a spatial arrangement, i.e., the neurons are typically organized in a line or a plane (see Figure 2.15). This kind of network is able to encode proximity features among the data. In other words, nodes that are neighbors in the network encode patterns that are adjacent in some sense in pattern space. Indeed, a self-organizing map has the property of *topology preservation*, that is, nearby input patterns should activate nearby output neurons on the map. A network that performs such a mapping is also called a *feature map*. In self-organized maps the weights of locally interacting units

are modified in response to input vectors according to a certain rule until a global ordering emerges.

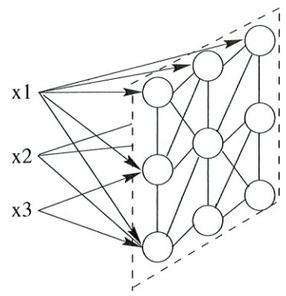


Figure 2.15
Self-Organizing Map.

An early model of self-organization was developed by Willshaw and von der Malsburg in the mid 1970s [235]. They used excitatory lateral connections with neighboring neurons, and inhibitory connections with distant neurons, instead of using a strict winner-take-all mechanism, as shown in Figure 2.16. The function that defines the form of such lateral connection weights is known as the *Mexican hat.*

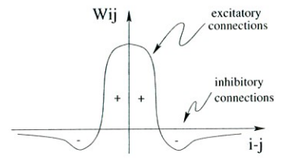


Figure 2.16 Mexican hat form of the lateral connection weights.

Teuvo Kohonen [114] described a self-organizing map learning algorithm that approximates the effect of a Mexican hat form of lateral connection weights by taking into account neighboring units for each output neuron. Neighborhoods introduce the notion of topology in neural learning and make the neurons' position with respect to each other a significant aspect of the net. The neighborhood can be square, rectangular, circular, etc. During training, all weight vectors associated with the winner neuron and its neighbors are updated.

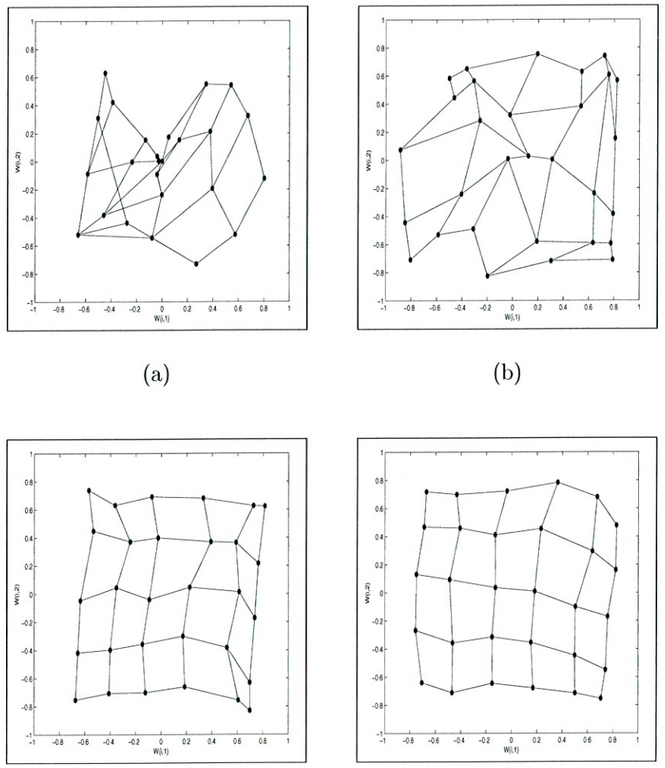
SOM can be seen as an extension to the simple competitive network of the previous section. Given an input vector $\mathbf{x}$, the winner unit is determined in the same way as in Equation 2.14. Weight adaptation is then performed according to a learning rule very similar to the competitive rule of Equation 2.15:

$$\Delta w_{ij} = \begin{cases} \rho N(i)(x_j - w_{ij}) & \text{for all units in the neighborhood} \\ 0 & \text{otherwise.} \end{cases} \tag{2.16}$$

The difference with the simple competitive rule is the presence of a "neighborhood" function $N(i)$ of unit i. This function is usually symmetric and monotonically decreasing with the distance. At the beginnings of the learning process N defines a large neighborhood and all the units in the neighborhood are updated for any input vector $\mathbf{x}$. As time goes by and learning progresses, the neighborhood is shrunk down while the learning rate ρ is also being iteratively decreased in order to achieve convergence. With respect to competitive training, learning now takes place over an extended neighborhood and it is this feature that allows the emergence of a topographic map. Eventually, a topographic map will be obtained in which nodes not only represent clusters of data but they are also arranged in the map such that neighboring units encode clusters that are "close" to each other in some sense in the input pattern space.

An example of this process is a two-dimensional self-organizing map of 30 neurons used to classify a certain number of random two-dimensional input vectors between -1 and 1. The two-dimensional map is a five by six neurons grid. Since each unit has only two inputs, it is possible to visualize the representation of this net in weight space, in which a point corresponds to each node's weight vector. The progressive organization of the weights according to the topology of the input space is depicted in Figure 2.17. At the beginning the weight vectors are randomly placed. However, after about 300 cycles one can see that the map has begun to organize itself and, as time passes and the neighborhood shrinks, the map is more and more evenly distributed across the input space. Finally, after 1000 cycles, the net has become ordered with the correct topology. Since the input vectors covered the square uniformly, the final map is correspondingly an almost regular grid in this domain. In this sense it can be said that the map has been able to "learn" the topology of the input space.

Feature maps have been applied in many areas including speech recognition, motor control, robot sensing, finance, data compression, and combinatorial optimization.



(a) (b)

(c) (d)

Figure 2.17 A 6×5 planar array of units mapping input vectors uniformly distributed in the interval [-1,1] . (a) After 100 iterations. (b) After 300 iterations. (c) After 500 iterations. (d) Final configuration after 1000 iterations. These images have been obtained with the Matlab software.

2.7 Fault Tolerance

SOFT-COMPUTING methodologies enjoy a certain amount of robustness in the face of errors, and this aspect is one of their strong points when applied to noisy and imprecise environments. Although ANNs are inspired by the nervous system, we have seen that they are but a pale imitation of actual biological neural networks. However, they retain some of the outstanding features of the latter such as collective computation abilities and resilience with respect to noise and errors. The fault-tolerant aspects of neural networks are all the more appreciable if one consider how faults may affect classical computing devices. Many computational procedures are strictly deterministic: given certain input values, they will always output the same results, using the same state transitions. This fact is comforting but what happens if errors appear somewhere? Change a single bit in that lovely fast sorting program and chances are that it won't work anymore. The fact is that engineered systems, as well as classical algorithms are built in such a way that errors always have serious or fatal consequences, unless much redundancy is built into the system. Thus, the standard engineering design process produces systems that are optimized in some sense but that are also very fragile.

In contrast, the human brain, in spite of its enormous complexity is much more reliable and less prone to catastrophic failure. Individual neurons are constantly dying without being replaced but the brain manages to compensate for this loss during lifetime, unless the damage is a serious one. Natural systems have not been designed in the customary engineering way; rather, they are the result of the evolutionary process (see Chapter 1). As such, they have been able to survive in all kinds of hostile environments by selecting and reproducing the structures that were the fittest at a given moment. At the same time, new structures were being created continuously by mutation and recombination and tried in new environments. These largely random processes have been remarkably successful, although they have taken millions of years. The human brain is as much the product of evolution as it is of learning. In fact, the detailed "wiring" of the brain is too complex to be determined by the genetic code alone. Together with nerve cell maturation and morphogenesis, learning shapes and fine-tunes the cell connections, allowing the brain to become the marvelous machine that allows us to function

properly in a continuously changing and complex environment. We will see in subsequent chapters that ANNs too can be designed by similar processes of evolution and learning, but these processes will obviously be artificial and usually simulated by a computer program. Artificial neural networks too can withstand a certain amount of errors and units may fail without necessarily driving the whole system to a grinding halt. This is also called *graceful degradation* and is obviously a very desirable property for machines as well as for humans. Most of the ANN systems that have been described in this chapter are not very sensitive to small amounts of noise or the loss of a unit here and there. Weight noise and unit misfiring has even been used purposefully during ANN training in order to enhance the fault-tolerance properties of the nets [48]. If the artificial neural system is a physical machine and not just a simulation program, this feature is even more useful, since failures are possible and very real in this case.

2.8 Artificial Neural Nets and Statistics

In many fields such as social science, finance, and economy there often exist a lot of data but explanatory theories and models are either lacking or they are not realistic enough as to guide prediction and decision. In data-rich but model-poor environments, people normally use statistical inference techniques to build system understanding and to estimate models. Parametric and non-parametric linear and non-linear regression models, such as autoregressive moving average (ARMA) models are well-known examples of statistical inference methods. Non-linear, nonparametric methods in particular make few *a priori* assumptions about the underlying system and, as a consequence, have a considerable number of degrees of freedom and explore a large space of functions to fit the observations.

It is clear that there are close links between neural network models and these statistical analysis techniques. Table 2.1 gives a minimal glossary of terms describing similar concepts in the two fields.

Indeed, from the formal statistical point of view, ANN models such as supervised feedforward nets can be seen as non-linear regresssion models [44], while other models are similar or identical to clustering, discriminant analysis techniques, or to stochastic approximation theory. Thus, the question is: are these ANN models really new and

Neural Networks	Statistics
learning	model estimation
supervised learning	non-linear regression
unsupervised learning	cluster analysis
weights	parameters
inputs	independent variables
outputs	dependent variables

Table 2.1 Artificial neural networks and statistics: glossary of corresponding terms.

what do they have to offer with respect to established statistical methods?

The answer (see [44] and the comments by other authors therein) is not entirely clear. As noted, it appears that, indeed, ANN formalisms can be reduced for the most part to some known statistical model. But for sure, ANNs are "sexier" and more attractive than equivalent statistical techniques for many non-specialists. Also, statisticians have concentrated for the most part on linear models and on comparatively small typical numbers of parameters. Neural networks provide us with tractable multivariate non-linear models that are easy to implement and that enjoy suggestive pictorial interpretations, avoiding the somewhat "dry" aspects of an equivalent statistical approach. There are other advantages as well: they are simple to implement, they enjoy a great simplicity of representation, and can be easily tuned to particular problems. Besides, they can be implemented in hardware with the obvious performance gains due to the direct execution and their intrinsic parallelism. Another advantage is that ANNs can be used as "modules" in *hybrid* systems. Statistics, on the other hand, may give ANNs researchers a firm footing for evaluating and analysing new ANN techniques and for relieving somewhat the lack of explanatory power of ANNs. Neural networks and statistics should not be seen as competing methods. Combinations of ANN and purely statistical methodologies are of great interest and are being actively pursued [44].